

Hola Profe, cómo estás?

Le cuento un poco cómo fue el proceso de creación de este proyecto, desde la idea inicial hasta los detalles técnicos finales. Mi objetivo era construir un sistema que no solo "haga cosas", sino que siga las buenas prácticas que estuvimos viendo tanto en clases, como las que actualmente estoy practicando en la facultad. Por ej, me gustó mucho incluir los principios SOLID, y Patrones de diseño (un tema que personalmente me gusta mucho), mas allá que no se requería hacerlo, me parece que suma mucho.

*- La Planificación

El desarrollo lo organicé usando un tablero Kanban en GitHub Projects. Me sirvió para no marearme y saber qué funcionalidad seguía. Además, usé una estrategia de ramas (Feature Branching) para simular un entorno profesional: nunca trabajé directo sobre la rama main, sino que fui creando ramas pequeñas para cada tarea (como feature/menu-interactivo o feature/excepciones), las cuales fui realizando PR hacia la rama "de producción" llamada develop. Y por último, esta rama develop, la fusioné con main, a través de un release para generar el producto final.

Si bien, el proyecto no pedí esto, a mí me sirvió mucho para ir simulando entornos de desarrollos reales, y para tener el proyecto sumamente organizado.

*- La Arquitectura

Modelos: Creé una jerarquía de herencia. Todos son Productos, pero las Bebidas tienen tamaño y temperatura, mientras que las Comidas tienen validaciones para celíacos, vegetarianos o veganos. Lo mismo hice con Personas, separando Clientes de Empleados.

Si bien aún no he utilizado la clase Empleado, me sirve para planificar una escalabilidad a futuro, donde los clientes incluso tienen roles (barista, cajero, supervisor), como también para siempre practicar herencia.

Servicios e Interfaces: Esta es la parte de la que estoy más orgulloso. No hice las clases sueltas, sino que primero definí Interfaces (IProductoService, IPedidoService). Esto me permitió separar el "qué debe hacer el sistema" de "cómo lo hace" realmente.

*-Principios SOLID

SRP --> clases como Producto o Cliente solo se representan a sí mismas, pero no guardan datos en la permanencia, ni validan stock.

OCP --> como mencioné, el sistema lo fui pensando para que sea escalable, y que el día de mañana pueda agregar nuevos productos(abierto a extensión), sin tener que romper o modificar clases existentes (cerrado para modificaciones).

LSP --> al crear la clase abstracta PProducto, pude generar una lista de pedidos que acepte tanto bebidas, como comidas, ya que la lista acepta productos, sin distinguir entre ellos.

y mi principio SOLID favorito:

Dependency Inversion Principle --> mi programa depende de abstracciones (las interfaces), no de clases concretas.

*- Patrones de Diseño

Singleton: Usé este patrón en los servicios para que, cuando el usuario navega por el menú, la lista de productos y el historial de pedidos sean siempre los mismos y no se reinicien al cambiar de opción. Evitando que en un futuro se cree una segunda instancia "por error" de estas clases que deberían ser únicas en el sistema.

*- Excepciones Personalizadas

La intención era crear e implementar varias excepciones personalizadas, pero por falta de tiempo no llegué a realizar las que tenía en mente. De modo que me limité a crear dos, muy básicas (StockInsuficiente y ProductoNoEncontrado) para al menos cumplir con la consigna. Pero seguiré trabajando en el proyecto para poder terminar estos detalles.

*- La Experiencia del Usuario (UI)

Para que no sea una consola aburrida de leer, personalicé todos los métodos toString(). Usé String.format para que todo salga alineado en columnas, como una tabla real. Al final de cada compra, el sistema genera un Ticket de Venta detallado que muestra los datos del cliente, los productos con sus subtotales y el total final de la operación.

Incluso incluí un método que simula un "arqueo de caja" al imprimir todos los pedidos realizados, 1 por 1, y al final mostrar el precio final de todas esas ventas.

*- Conclusión

El proyecto me sirvió para bajar a tierra conceptos complejos como el polimorfismo, herencia, UML de clases, e incluso la inversión de dependencias. Siento que quedó una base sólida sobre la cual se podría agregar una base de datos real o una interfaz gráfica en el futuro.

Espero sea de tu agrado. Cualquier comentario, observación o crítica constructiva, va a ser muy bien recibida.

Desde ya, muchísimas gracias por todo, y un saludo enorme.

Orellana, Gabriel Alejandro

DNI 36087700

COMISION JAVA - 26138